

User Stress Report - NanoServe Load Testing

User Stress Report - NanoServe Load Testing

****Date:**** 2026-07-25

****Tool:**** `tests/load\_test\_report.py`

****Device:**** CPU (AVX2)

****Verdict:**** PASSED AT ALL TIERS

Executive summary

Concurrent HTTP load tests simulate real users hitting `/v1/completions` simultaneously. All three tiers - **50, 150, and 300 users

Test methodology

1. Start server with appropriate scaling profile (see deployment column).
2. Run `python3 tests/load\_test\_report.py --preset {50 150 300}`.
3. Each virtual user sends one completion request with `max\_tokens=32`, `device=cpu`.
4. Metrics: success rate, throughput (req/s), latency percentiles.

Results - 50 users (baseline)

Suitable for development and demos.

Results - 150 users

Acceptable for moderate load on a single process; consider production script for headroom.

Results - 300 users (non-Docker)

Raw JSON: `documentation/load\_300.json`

Recommended 300-user production (non-Docker)

./install.sh

source .venv/bin/activate && source .env.nanoserve

./scripts/run_native_300.sh # nginx :8000 + 4× gunicorn, workers = nproc/4 each

This matches Docker-free scaling documented in [SCALING.md](./SCALING.md) and splits engine workers across processes to

Comparison chart

How to reproduce

source .venv/bin/activate && source .env.nanoserve

./scripts/run_native.sh & # or run_native_300.sh for 300-tier prod

sleep 2

python3 tests/load_test_report.py --preset 300

Conclusion

NanoServe handles 300 simultaneous users on CPU with full success. Latency increases with concurrency as expected for a